

Problems on Binary Tree

DAY 172
29/12/25

Question 1: Give an Algorithm for finding maximum element In a Binary Tree

One Simple Way of solving this problem is: find the maximum element in the left subtree, find the maximum element in the right subtree, compare them with root data and select the one which is giving the maximum value.

This approach can be easily implemented with recursion

Algorithm

- ① if tree is empty $\rightarrow$ return $-\infty$
- ② Recursively find
 - max in left subtree
 - max in right subtree
- ③ Compare:
 - root value
 - left max
 - right max
- ④ Return the largest

```
public static int findMax (TreeNode root) {
```

```
    if (root == null) {
```

```
        return Integer.MIN_VALUE;
```

```
    }
```

```
    int data = root.data;
```

```
    int maxL = findMax (root.left);
```

```
    int maxR = findMax (root.right);
```

```
    return Math.max (data, Math.max (maxL, maxR));
```

```
}
```

code

Question 2 Give an algorithm for finding the maximum element in Binary tree without recursion

Solution Using level order traversal; just observe the element's data while deleting.

Algorithm

```
if tree is Empty, return  $-\infty$ 
create an Empty Queue
Insert the Root into the Queue
Initialize  $\text{max} = -\infty$ 
While the queue is not Empty:
    • Remove a node from the Queue
    • Compare its data with  $\text{max}$ 
    • Update  $\text{max}$  if current data is larger
    • Insert left and right children into the queue (if present)
    • return  $\text{max}$ .
```

```
static int findmax (TreeNode root) {
```

```
    if (root == null)
        return Integer.MIN_VALUE;
```

```
    Queue < TreeNode > q = new LinkedList < > ();
    q.add (root);
```

```
    int max = Integer.MIN_VALUE;
```

```
    while (!q.isEmpty()) {
```

```
        TreeNode cur = q.poll();
```

```
        if (cur.data > max)
            max = cur.data;
```

```
        if (cur.left != null)
            q.add (cur.left);
        if (cur.right != null)
            q.add (cur.right);
    }
```

code

```
q.add (cur.left);
```

```
if (cur.Right != null)  
    q.add (cur.right);
```

```
    }  
    return max;  
}
```